Rapidjson parser

```
#pragma once
#include <chrono>
#include <iostream>
#include <fstream>
#include <rapidjson/writer.h>
#include <sstream>
#include <string>
#include <vector>
#include <array>
#include "rapidjson/document.h"
#include "rapidjson/stringbuffer.h"

struct Queue_t {
  uint32_t index;
  uint32_t tag;
  uint32_t sm;
  bool active;
  bool mode_switch;
  uint32_t az_code;
  uint32_t el_code;
  uint32_t fr_code;
};

struct Overview_t {
  struct PacketPulses_t {
    bool steering_ctrl;
    bool prb_disable;
    bool orthogonal;
    bool active;
    bool qcw;
    uint32_t bid;
    uint32_t index;
    uint32_t nor;
    double prp_us;
    double phase_deg;
    uint32_t tag;
  };

  uint32_t mid;
  std::string desc;
  std::vector<PacketPulses_t> packet_of_pulses;
};

struct PulseBursts_t{
  struct PilotBurst_t {
    bool active;
    bool sap;
    double amplitude_dB;
```

```
    double doppler_kHz;
    uint32_t index;
    double pause_us;
    uint32_t sid;
    double width_us;
  };

  struct ProbeBurst_t {
    bool active;
    double fpf_ns;
    double rpf_ns;
    double if_MHz;
    uint32_t index;
    double pause_us;
    uint32_t sid;
    double width_us;
    bool window;
  };

  uint32_t bid;
  std::string desc;
  std::vector<PilotBurst_t> pilot_burst;
  std::vector<ProbeBurst_t> probe_burst;
};

struct SignalIds_t {
  struct Signal_t {
    double scale;
    std::vector<double> amplitude_coefs;
    std::vector<double> phase_coefs;
    std::string desc;
  };

  struct XCorr_t {
    double scale;
    std::vector<double> amplitude_coefs;
    std::vector<double> phase_coefs;
  };

  struct GaussWindow_t {
    double mean;
    double sigma;
    double hold;
    double stop;
  };

  struct Weight_t {
    double scale;
    std::vector<double> filter_coefs;
  };

  struct Ivm_t {
    double pause_us;
    double width_us;
```

```
    double start_pause_us;
    double start_width_us;
  };

  uint32_t sid;
  Signal_t signal;
  XCorr_t xcorr;
  GaussWindow_t gauss_w;
  Weight_t weight;
  Ivm_t ivm;
};

class CfgJsParser {
  public:
    CfgJsParser(const char* config_path){
      std::ifstream cfg(config_path);
      if (!cfg.is_open()) {
        std::cerr << "Не удалось открыть файл!" << std::endl;
        return;
      }

      std::stringstream buffer;
      buffer << cfg.rdbuf();
      std::string json_content = buffer.str();
      cfg.close();

      parseAndSaveData(json_content);
    }

    void parseAndSaveData(std::string& json_content){
      rapidjson::Document doc;
      if (doc.Parse(json_content.c_str()).HasParseError()) {
        std::cerr << "Ошибка парсинга JSON! Код ошибки: "
                  << doc.GetParseError() << std::endl;
        return;
      }
      // На этом этапе имеем только дерево и строки, нужно получить данные

      if (doc.IsObject() && doc.HasMember("control"))
      {
        auto& ctrl = doc["control"];
        dsp_e = ctrl["dsp_enabled"].GetBool();
        pilot_e = ctrl["pilot_enabled"].GetBool();
        probe_e = ctrl["probe_enabled"].GetBool();
        overview_mode = ctrl["overview_mode"].GetUint();
        dsp_out_mux = ctrl["dsp_output_mux"].GetString();
        sync_pulse_mux = ctrl["sync_pulse_mux"].GetString();
        tx_output_mux = ctrl["tx_output_mux"].GetString();
        in_mux_rx[0] = ctrl["input_mux_rx0"].GetString();
        in_mux_rx[1] = ctrl["input_mux_rx1"].GetString();
        in_mux_rx[2] = ctrl["input_mux_rx2"].GetString();
        in_mux_rx[3] = ctrl["input_mux_rx3"].GetString();

        auto& tasks = doc["tasks"];
```

```
        const auto& que_array = tasks["que"];
        for (auto it = que_array.Begin(); it != que_array.End(); ++it) {
          ques.push_back(Queue_t{
            .index = (*it)["index"].GetUint(),
            .tag = (*it)["tag"].GetUint(),
            .sm = (*it)["sm"].GetUint(),
            .active = (*it)["active"].GetBool(),
            .mode_switch= (*it)["mode_switch"].GetBool(),
            .az_code = (*it)["az_code"].GetUint(),
            .el_code = (*it)["el_code"].GetUint(),
            .fr_code = (*it)["fr_code"].GetUint()
          });
        }

        auto& om = doc["overview_modes"];
        for (auto it = om.Begin(); it != om.End(); ++it){
          Overview_t ovm;
          ovm.mid = (*it)["mid"].GetUint();
          ovm.desc = (*it)["description"].GetString();
          auto& pop = (*it)["packet_of_pulses"];
          for(auto pulses = pop.Begin(); pulses != pop.End(); ++pulses)
          {
            Overview_t::PacketPulses_t pp;
            pp.steering_ctrl = (*pulses)["steering_ctrl"].GetBool();
            pp.prb_disable = (*pulses)["prb_disable"].GetBool();
            pp.active = (*pulses)["active"].GetBool();
            pp.bid = (*pulses)["bid"].GetUint();
            pp.index = (*pulses)["index"].GetUint();
            pp.nor = (*pulses)["nor"].GetUint();
            pp.orthogonal = (*pulses)["orthogonal"].GetBool();
            pp.prp_us = (*pulses)["prp_us"].GetDouble();
            pp.phase_deg = (*pulses)["phase_deg"].GetDouble();
            pp.qcw = (*pulses)["qcw"].GetBool();
            pp.tag = (*pulses)["tag"].GetUint();
            ovm.packet_of_pulses.push_back(pp);
          }
          overview_modes.push_back(ovm);
        }
      }

      auto& pulse_bur = doc["pulse_bursts"];
      for (auto it = pulse_bur.Begin(); it != pulse_bur.End(); ++it){
        PulseBursts_t pb;
        pb.bid = (*it)["bid"].GetUint();
        pb.desc = (*it)["description"].GetString();

        auto& pilot_burst = (*it)["pilot_burst"];
        for (auto pilot = pilot_burst.Begin(); pilot != pilot_burst.End();
++pilot){
          PulseBursts_t::PilotBurst_t pilot_b;
          pilot_b.active = (*pilot)["active"].GetBool();
          pilot_b.amplitude_dB = (*pilot)["amplitude_dB"].GetDouble();
          pilot_b.doppler_kHz = (*pilot)["doppler_kHz"].GetDouble();
          pilot_b.index = (*pilot)["index"].GetUint();
```

```
          pilot_b.pause_us = (*pilot)["pause_us"].GetDouble();
          pilot_b.sid = (*pilot)["sid"].GetUint();
          pilot_b.sap = (*pilot)["sap"].GetBool();
          pilot_b.width_us = (*pilot)["width_us"].GetDouble();
          pb.pilot_burst.push_back(pilot_b);
        }

        auto& probe_burst = (*it)["probe_burst"];
        for (auto probe = probe_burst.Begin(); probe != probe_burst.End();
++probe){
          PulseBursts_t::ProbeBurst_t probe_b;
          probe_b.active = (*probe)["active"].GetBool();
          probe_b.fpf_ns = (*probe)["fpf_ns"].GetDouble();
          probe_b.rpf_ns = (*probe)["rpf_ns"].GetDouble();
          probe_b.if_MHz = (*probe)["if_MHz"].GetDouble();
          probe_b.index = (*probe)["index"].GetUint();
          probe_b.pause_us = (*probe)["pause_us"].GetDouble();
          probe_b.sid = (*probe)["sid"].GetUint();
          probe_b.width_us = (*probe)["width_us"].GetDouble();
          probe_b.window = (*probe)["window"].GetBool();
          pb.probe_burst.push_back(probe_b);
        }
        pulse_bursts.push_back(pb);
      }

      auto& sig_ids = doc["signal_ids"];
      for (auto it = sig_ids.Begin(); it != sig_ids.End(); ++it){
        SignalIds_t si;
        si.sid = (*it)["sid"].GetUint();

        auto& sig = (*it)["signal"];
        SignalIds_t::Signal_t s;
        s.scale = sig["scale"].GetDouble();

        for(auto ac = sig["ampltude_coefs"].Begin(); ac !=
sig["ampltude_coefs"].End(); ++ac)
          s.amplitude_coefs.push_back(ac->GetDouble());

        for(auto pc = sig["phase_coefs"].Begin(); pc !=
sig["phase_coefs"].End(); ++pc)
          s.phase_coefs.push_back(pc->GetDouble());

        s.desc = sig["description"].GetString();
        si.signal = s;

        auto& xc = (*it)["x_corr"];
        SignalIds_t::XCorr_t x;
        x.scale = xc["scale"].GetDouble();

        for(auto ac = xc["ampltude_coefs"].Begin(); ac !=
xc["ampltude_coefs"].End(); ++ac)
          x.amplitude_coefs.push_back(ac->GetDouble());

        for(auto pc = xc["phase_coefs"].Begin(); pc !=
```

```
xc["phase_coefs"].End(); ++pc)
          x.phase_coefs.push_back(pc->GetDouble());
        si.xcorr = x;


        auto& gw = (*it)["gauss_window"];
        SignalIds_t::GaussWindow_t g;
        g.mean =  gw["mean"].GetDouble();
        g.hold =  gw["hold"].GetDouble();
        g.sigma = gw["sigma"].GetDouble();
        g.stop =  gw["stop"].GetDouble();
        si.gauss_w = g;


        auto& wg = (*it)["weight"];
        SignalIds_t::Weight_t w;
        w.scale =  wg["scale"].GetDouble();

        for (auto fc = wg["filter_coefs"].Begin(); fc !=
wg["filter_coefs"].End(); ++fc)
          w.filter_coefs.push_back(fc->GetDouble());
        si.weight = w;

        auto& ivmjs = (*it)["ivm"];
        SignalIds_t::Ivm_t i;
        i.pause_us = ivmjs["pause_us"].GetDouble();
        i.width_us = ivmjs["width_us"].GetDouble();
        i.start_pause_us = ivmjs["start_pause_us"].GetDouble();
        i.start_width_us = ivmjs["start_width_us"].GetDouble();
        si.ivm = i;

        signal_ids.push_back(si);
      }
    }

  private:
    bool dsp_e{false};
    bool pilot_e{false};
    bool probe_e{false};
    uint32_t overview_mode = 0;
    uint32_t dec_fact_code = 0;
    std::string dsp_out_mux;
    std::string sync_pulse_mux;
    std::string tx_output_mux;
    std::array<std::string, 4> in_mux_rx{};

    std::vector<Queue_t> ques;
    std::vector<Overview_t> overview_modes;
    std::vector<PulseBursts_t> pulse_bursts;
    std::vector<SignalIds_t> signal_ids;
};
```

Protobuf parser

```
#pragma once
#include <fstream>
#include <chrono>
#include <google/protobuf/text_format.h>
#include <google/protobuf/io/zero_copy_stream_impl.h>
#include "gen/dsp_test_sweep.pb.h"

class ConfigPbParser {
  public:
    ConfigPbParser(const char* config_path) : path(config_path){}

    void parseAndSaveDataBinaryProto(){
      cfg.open(path, std::ios::binary);
      if (!cfg.is_open()) {
        std::cerr << "Не удалось открыть файл!" << std::endl;
        return;
      }

      if (!main_cfg.ParseFromIstream(&cfg)) {
        std::cerr << "Ошибка парсинга .pb файла!" << std::endl;
        return;
      }

      // На этом этапе всё уже находится в main_cfg
    }

  private:
    const char* path;
    std::ifstream cfg;
    MainConfig main_cfg;
};
```